香港科技大學
THE HONG KONG UNIVERSITY OF SCIENCE AND TECHNOLOGY

ELEC6910J
Deep Reinforcement Learning

Lecture 13. Exam Review

Course Instructor: Ling PAN
Department of Electronic and Computer Engineering
Hong Kong University of Science and Technology

Most slides are adapted from UC Berkeley CS188, CMU 10-403, UCL COMPM050

Announcement

- In-class exam: 15:00-17:50, Oct 23, Room 6555
  - A one-page (single-sided) cheatsheet is allowed.
  - Calculator is allowed.
  - Please contact the TAs **in advance** is you have a time conflict.

ELEC6910J **Core Checklist for the Exam**

Time & Date: 15:00-17:40, Oct. 23 (in-class exam)
Venue: Room 6555 (our classroom)
Question types: multiple-choice, fill-in-the-blanks, calculations
Note: Closed-book exam; however, *you can bring one double-sided A4 sheet of notes*.

Bandits & Exploration

- ☐ The exploration vs. exploitation trade-off
- ☐ ϵ-greedy exploration strategy
- ☐ Stationary vs. non-stationary bandits

Markov Decision Process (MDP)

- ☐ Definition & understanding
- ☐ The role of the discount factor γ and its impact on optimal policies

Value Functions and Bellman Equations

- ☐ Recursive relationships between current and future values
- ☐ State-value functions and action-value functions
- ☐ Bellman expectation equations and Bellman optimality equations

Dynamic Programming

- ☐ Value Iteration: update equations and finding the optimal policy
- ☐ Policy Evaluation: iterative calculation of state values for a given policy
- ☐ Policy Iteration: policy evaluation and policy improvement steps

Monte Carlo Methods

- ☐ First/Every-visit Monte Carlo prediction and value estimation
- ☐ Characteristics of Monte Carlo methods

Temporal Difference Learning

- ☐ Calculation of the TD target and TD error (including multi-step TD)
- ☐ Monte Carlo vs. TD learning: fundamental differences, bias, and variance
- ☐ Differences between on-policy and off-policy learning approaches
- ☐ SARSA: On-policy control algorithm
- ☐ Q-learning: Off-policy control algorithm

General Machine Learning Concepts

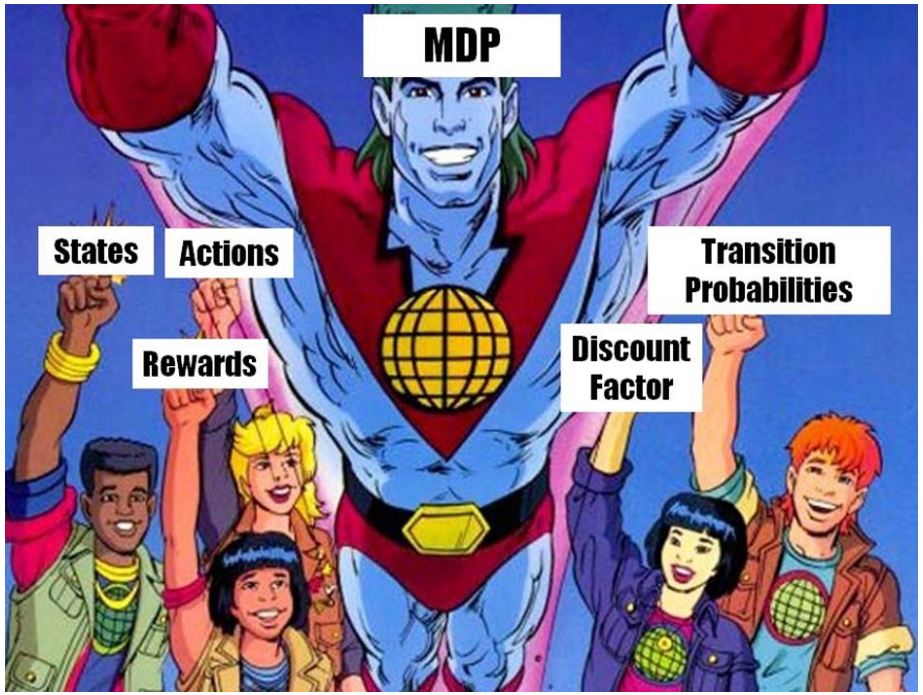
- ☐ Differences between supervised learning and reinforcement learning
- ☐ Differences among DP, MC, and TD

Practical Problem-Solving

- ☐ Step-by-step tabular value updates, deriving optimal policies
- ☐ Matrix and algebraic calculations

Markov Decision Process (MDP)

- Definition: (S, A, T, R, γ)
  - A set of states $s \in S$
  - A set of actions $a \in A$
  - A transition function $T(s, a, s')$
    - Probability that a from s leads to s', i.e., $P(s'|s, a)$
    - Also called the model or the dynamics
  - A reward function $R(s, a, s')$
  - A discount factor γ



- Quantities:
  - Policy: map of states to actions
  - Utility: sum of discounted rewards
  - State values: expected future utility from a state
  - State-action (Q-)values: expected future utility from a state-action pair

Value Functions

- **Recursive** relationships for returns

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots$$
$$= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots)$$
$$= R_{t+1} + \gamma G_{t+1}$$

Condition on a state and take expectations

$$\mathbb{E}_\pi[G_t | S_t = s] = \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} | S_t = s]$$

$$V^\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) | S_t = s]$$

$V^\pi(s)$: value of a state

$\mathbb{E}_\pi[R_{t+1}$: expected immediate reward

$+\gamma V^\pi(S_{t+1})$: +discounted value of next state

$| S_t = s]$: if the agent starts at state s

and uses the policy to choose its actions for all time steps

(law of total probability)

$$= \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V^\pi(s')]$$

Value Functions

- Bellman equation for V^{π}:

$$V^{\pi}(s) = \sum_{a} \pi(a|s) \sum_{s',r} p(s', r|s, a)[r + \gamma V^{\pi}(s')]$$

  - Relationship between the value of the current state and the values of its successor states
  - The value function $V^{\pi}(s)$ is the unique solution to its Bellman equation



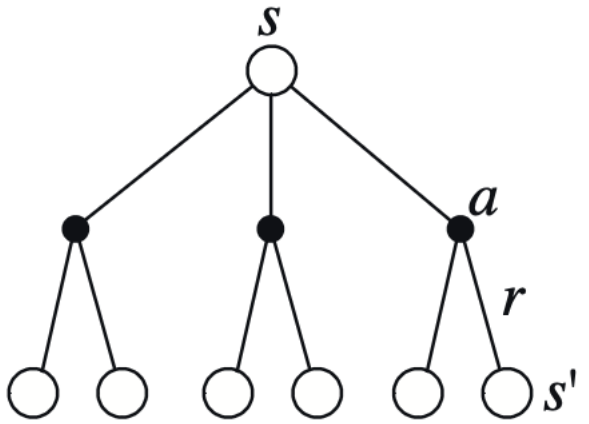
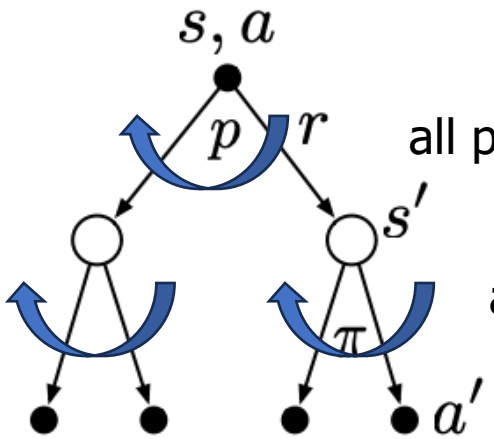


Figure. Backup diagram.

Value Functions

- Bellman equation for Q^π:

$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a)\left[r + \gamma \sum_{a'} \pi(a'|s')\, Q^\pi(s',a')\right]$$



all possible next states and rewards listed with their probabilities

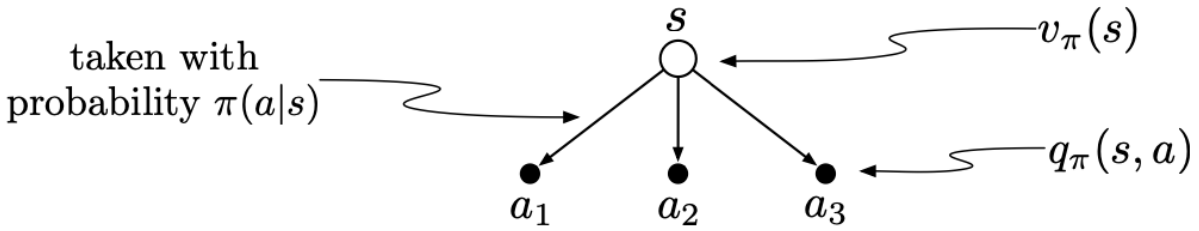
all next actions listed with their probabilities

Figure. Backup diagram.

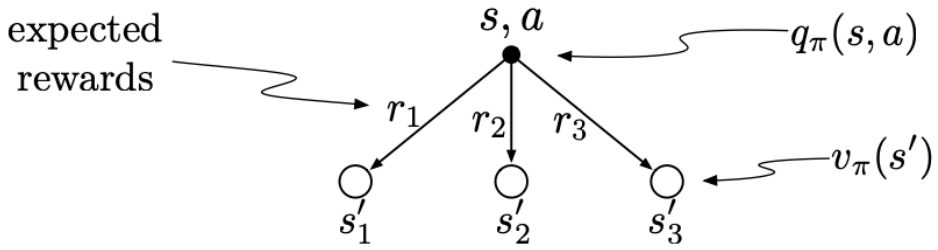
Value Functions

- Relating V^π and Q^π:

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s,a)$$



$$Q^\pi(s,a) = \sum_{s',r} p(s',r|s,a)[r + \gamma V^\pi(s')]$$



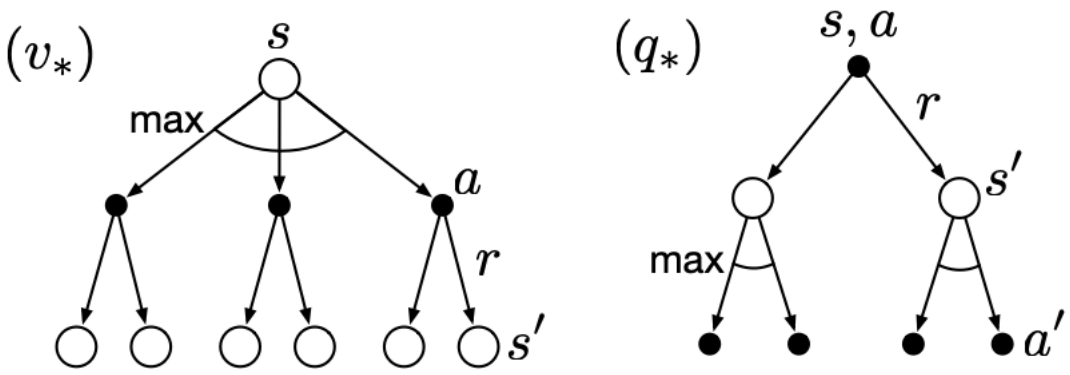
Optimal Value Functions

- Bellman optimality equation for V^*:
  - The value of a state under an optimal policy must equal the expected return for the best action from that state

$$V^*(s) = \max_a \left(\sum_{s',r} p(s', r|s, a)[r + \gamma V^*(s')] \right)$$

- Bellman optimality equation for Q^*:

$$Q^*(s, a) = \sum_{s',r} p(s', r|s, a) \left[r + \gamma \max_{a'} Q^*(s', a') \right]$$



$V^*(s)$ and $Q^*(s, a)$ is the unique solution of this system of nonlinear equations

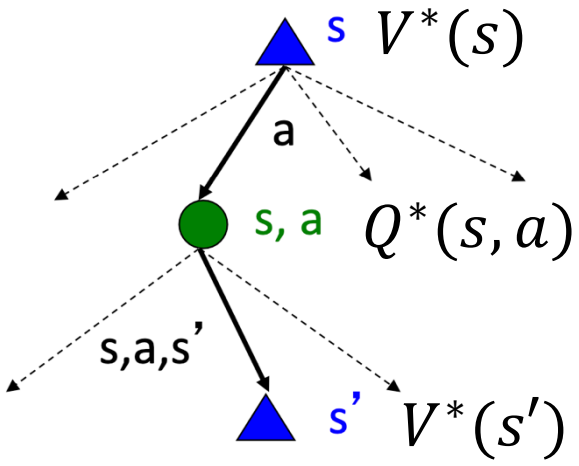
Optimal Value Functions

- Recursive definition of value (Bellman equation)
  - Optimal state value function:

$$V^*(s) = \max_a Q^*(s,a)$$

  - Optimal state-action value function:

$$Q^*(s,a) = \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$



- Optimal policy

$$\pi^*(s) = \operatorname{argmax}_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s,a)$$

Value Iteration

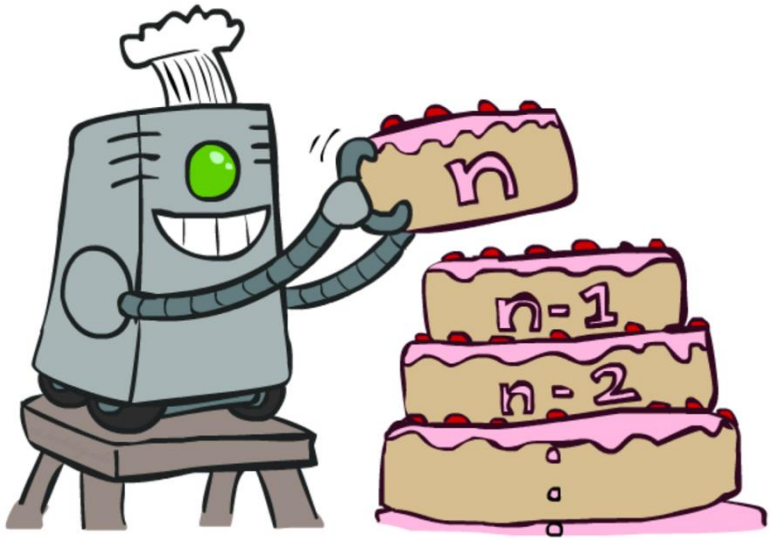
- Bellman equation characterizes the optimal values

$$V^*(s) = \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$

- Value iteration computes the optimal values

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$$

  - A fixed point solution method

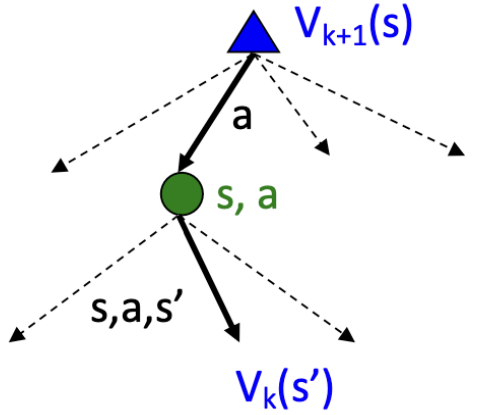


Value Iteration

- Start with $V_0(s) = 0$: zero utility if the game ends now
- Given vector of $V_k(s)$ values, we can compute $V_{k+1}(s)$

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$$

- Repeat until convergence, which yields V^*

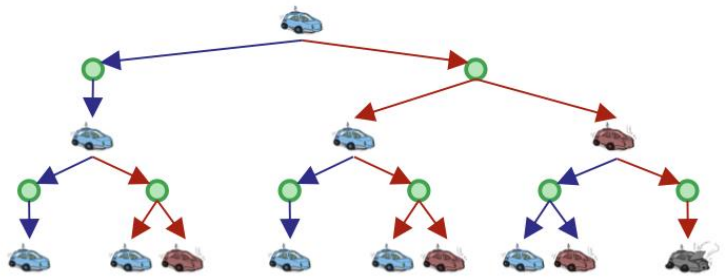


- The complexity of each iteration in terms of state space cardinality $|S|$ and action space cardinality $|A|$
  - $O(|S|^2|A|)$ – as you need to operate on every state

- Theorem: will converge to unique optimal values
  - Basic idea: approximations get refined towards optimal values
  - Policy may converge long before values do

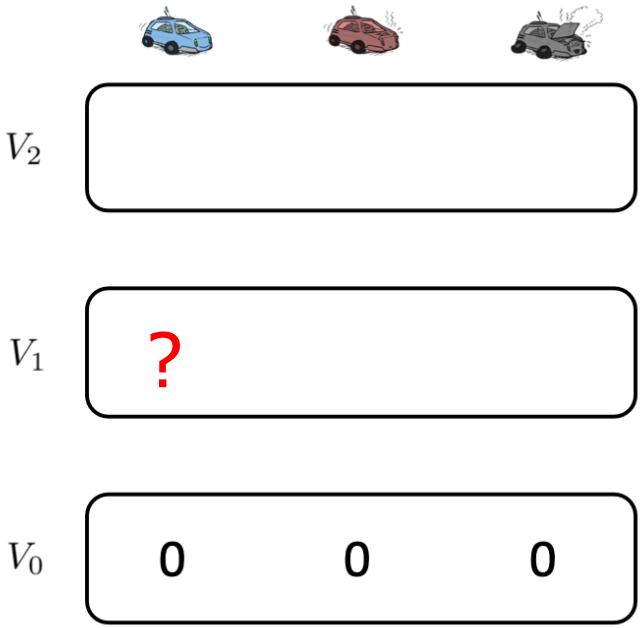
Value Iteration

- Example: Car racing

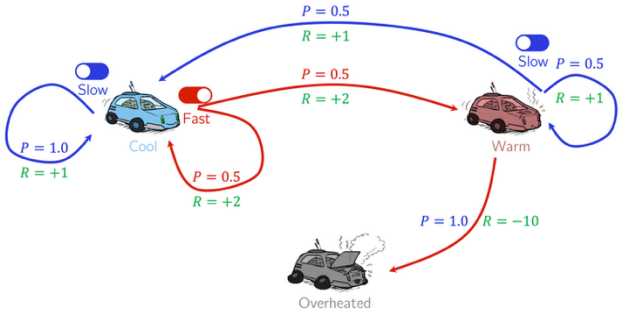


s	a	s'	$P(s'\|s,a)$	$R(s,a,s')$
Cool	Slow	Cool	1.0	+1
Cool	Fast	Cool	0.5	+2
Cool	Fast	Warm	0.5	+2
Warm	Slow	Cool	0.5	+1
Warm	Slow	Warm	0.5	+1
Warm	Fast	Overheated	1.0	-10
Overheated	(end)	Overheated	1.0	0

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')] \quad \text{(assume no discount)}$$



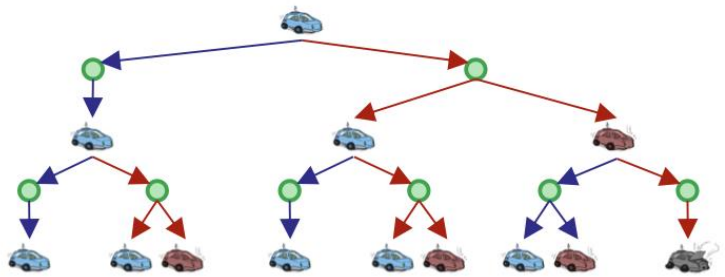
- action=slow:
  $1\times(1+0)=1$
- action=fast:
  $0.5\times(2+0)+0.5\times(2+0)=2$



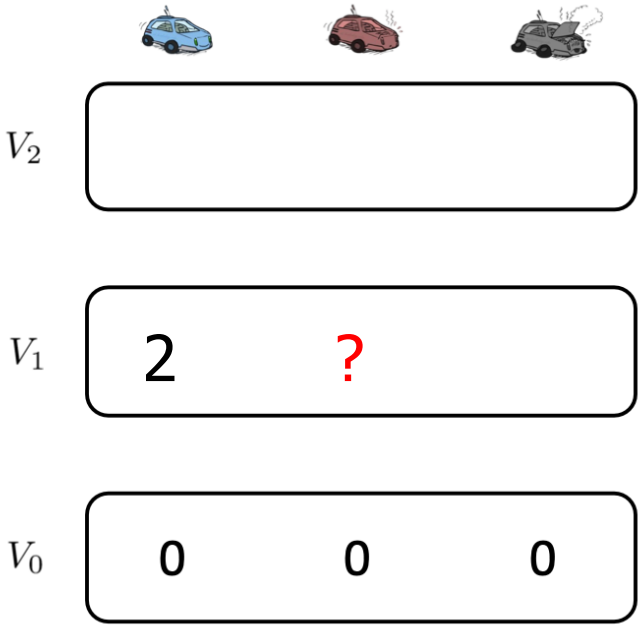
Value Iteration

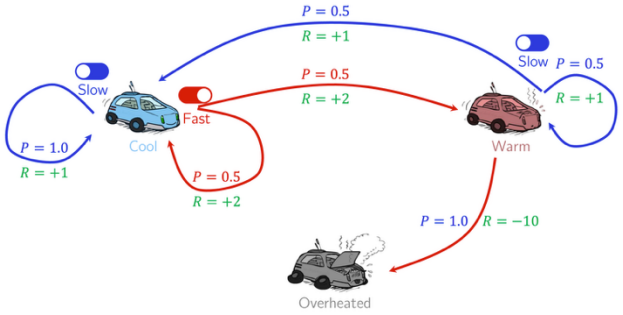
- Example: Car racing

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')] \quad \text{(assume no discount)}$$



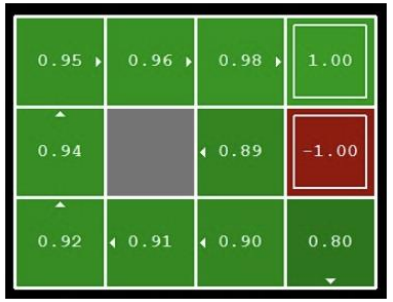
s	a	s'	$P(s'\|s,a)$	$R(s,a,s')$
Cool	Slow	Cool	1.0	+1
Cool	Fast	Cool	0.5	+2
Cool	Fast	Warm	0.5	+2
Warm	Slow	Cool	0.5	+1
Warm	Slow	Warm	0.5	+1
Warm	Fast	Overheated	1.0	-10
Overheated	(end)	Overheated	1.0	0





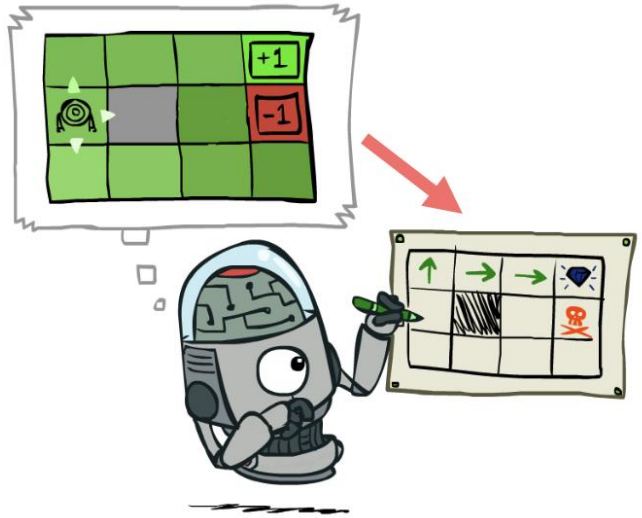
Computing Actions from Values

- Problem: how do we get actions?
  - Imagine we have the optimal values $V^*(s)$
  - How should we act?



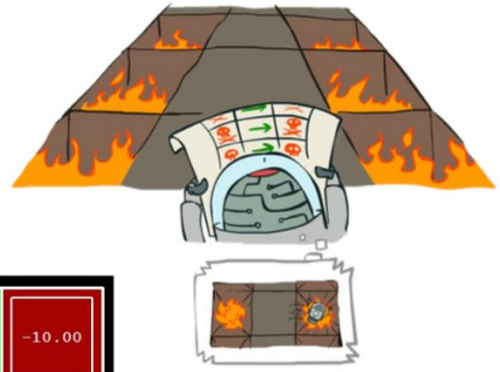
- Policy extraction: gets the policy implied by the values
  - We need to do a one-step lookup

$$\pi^*(s) = \text{argmax}_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V^*(s')]$$



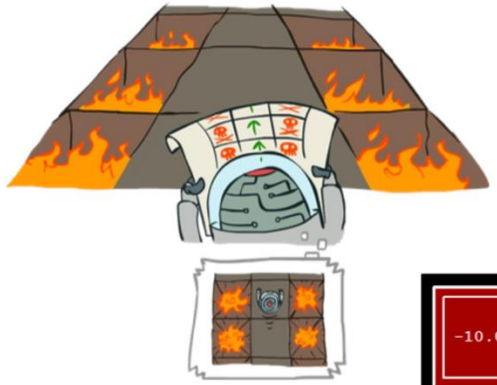
Policy Evaluation

Always Go Right





Always Go Forward





Usage: computing the value function for a fixed policy is to help find better policies.

$$V_0^\pi(s) = 0$$

$$V_{k+1}^\pi(s) \leftarrow \sum_{s'} T(s, \pi(s), s')[R(s, \pi(s), s') + \gamma V_k^\pi(s')]$$

- Efficiency: $O(|S|^2)$ per iteration

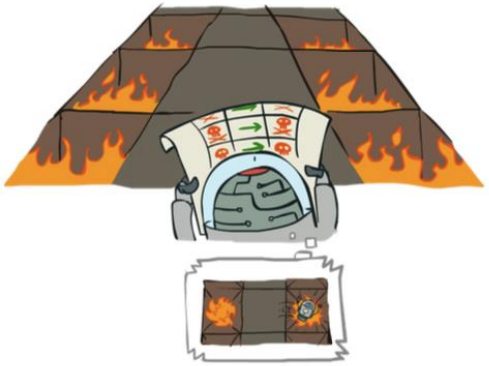
Policy Improvement

- For a given state s,

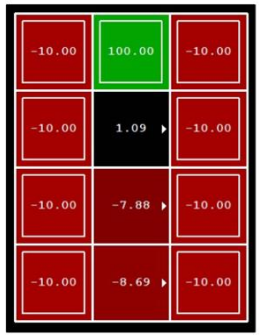
  would it be better to do an action $a \neq \pi(s)$ and thereafter follow π?

  - We know how good it is to follow the current policy π from $s \rightarrow V^{\pi}(s)$
  - Would it be better or worse to change to a new policy?

Policy π: Always Go Right



Value function V^{π}



New policy:

- Go forward
- Follow π (always go right) thereafter





Policy Iteration

- Evaluation: For fixed current policy π, find values with policy evaluation:
  - Iterate until values converge:

$$V_{k+1}^{\pi_i}(s) \leftarrow \sum_{s'} T(s, \pi_i(s), s') \left[R(s, \pi_i(s), s') + \gamma V_k^{\pi_i}(s') \right]$$

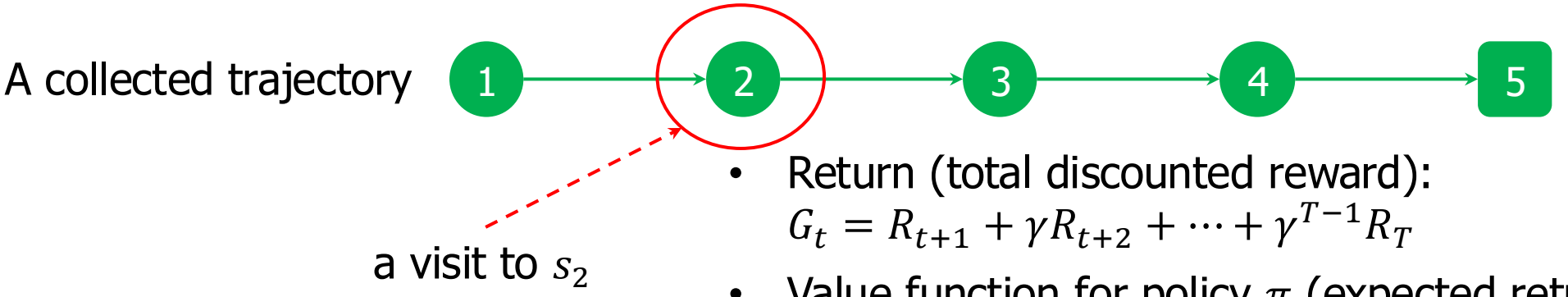
  - End up with value function
- Improvement: For fixed values, get a better policy using policy extraction
  - One-step look-ahead:

$$\pi_{i+1}(s) = \arg\max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^{\pi_i}(s') \right]$$

- Repeat steps until policy converges

Monte-Carlo Prediction

- Goal: learn $V^{\pi}(s)$ from episodes of experience under policy π
- Method: average returns observed after visits to s



- Return (total discounted reward):
  $G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-1} R_T$
- Value function for policy π (expected return):

  $V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$ starting at state s

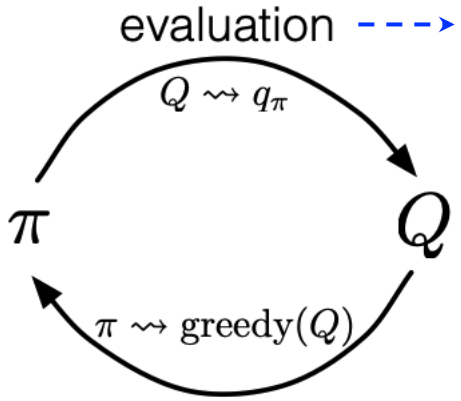
  value of a state — expected discounted return
- Monte-Carlo policy evaluation uses empirical mean return instead of expected return

- First-visit MC: average returns only for first time s is visited in an episode
- Every-Visit MC: average returns for every time s is visited in an episode

Monte-Carlo Control

- Consider a Monte-Carlo version of classical policy iteration

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*$$



MC policy iteration step:
Policy evaluation using MC methods
(followed by policy improvement)

Policy improvement step:
Greedify with respect to Q-function

$$\pi(s) = \text{argmax}_a Q(s,a)$$

- Meaning: for each s, deterministically chooses an action with maximal Q-value
- Effect: construct each π_{k+1} as the greedy policy w.r.t. Q_k^π

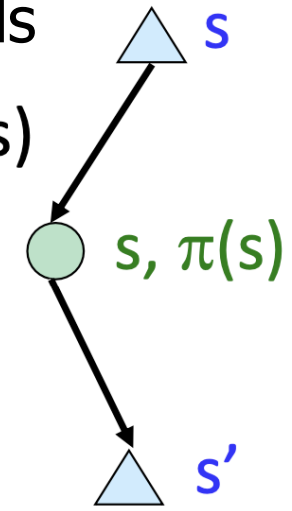
$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg\max_a q_{\pi_k}(s,a)) \\ &= \max_a q_{\pi_k}(s,a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \end{aligned}$$

Temporal Difference Prediction

$$V(S_t) \leftarrow V(S_t) + \alpha[\overbrace{R_{t+1} + \gamma V(S_{t+1}) - V(S_t)}^{\text{TD error } \delta_t}]$$

$R_{t+1} + \gamma V(S_{t+1})$: TD target: an estimate of the return

- ✓ TD methods learn directly from episodes of experience
- ✓ TD is model-free: no knowledge of MDP transitions / rewards
- ✓ TD learns from incomplete episodes, by bootstrapping
- ✓ TD updates a guess towards a guess



Temporal Difference Prediction

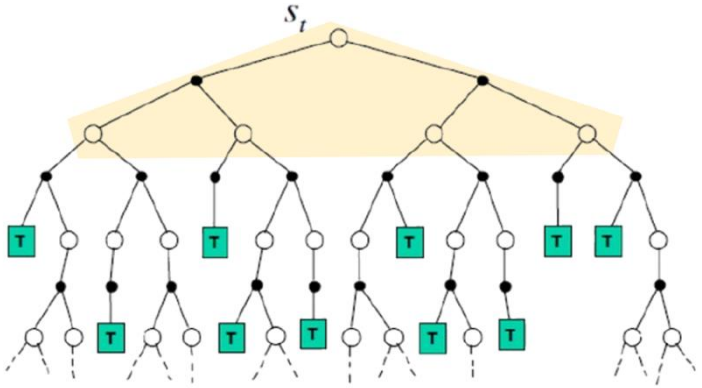
MC:
- The expected value is unknown
- A sample return is used in place of the real expected return

$$
\begin{aligned}
v_\pi(s) &\doteq \mathbb{E}_\pi[G_t \mid S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]
\end{aligned}
$$

TD: combine the sampling of MC and bootstrapping of DP
- Sample the expected value
- Use a current estimate $V(S_{t+1})$ of the true unknown $v_\pi(S_{t+1})$

DP:
- The expected values are provided by a model
- Use a current estimate $V(S_{t+1})$ of the true unknown $v_\pi(S_{t+1})$

Comparison

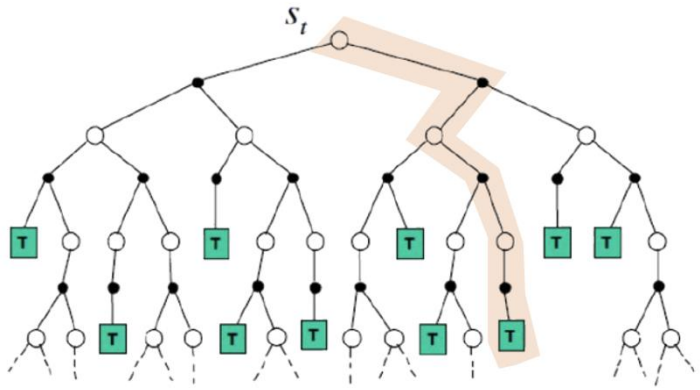


bootstrapping

Dynamic Programming

$V(S_t) \leftarrow \mathbb{E}_\pi[R_{t+1} + \gamma V(S_{t+1})]$

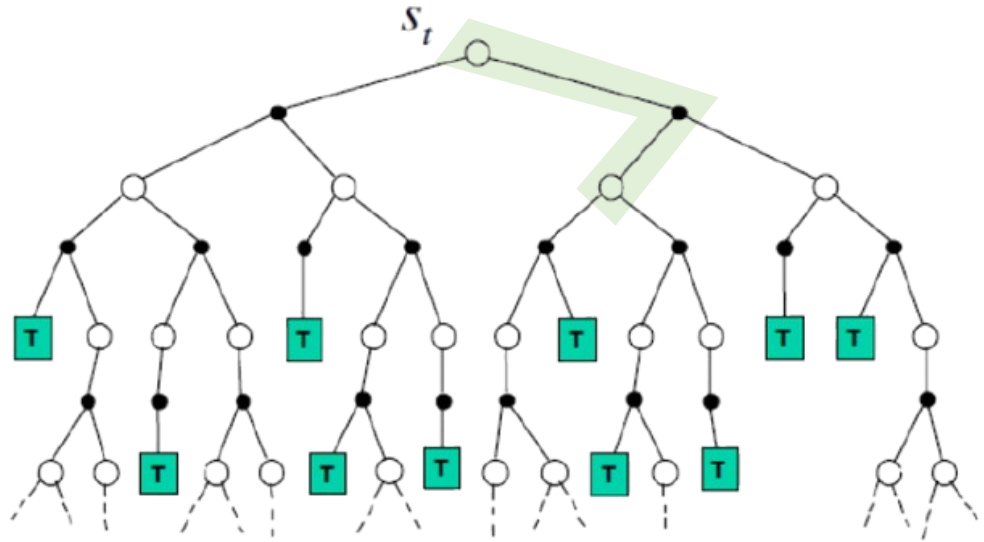
$= \sum_a \pi(a|S_t) \sum_{s',r} p(s', r|S_t, a)[r + \gamma V(s')]$



sampling

Monte-Carlo Methods

$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$ (every visit)



Temporal Difference (TD (0))

$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$

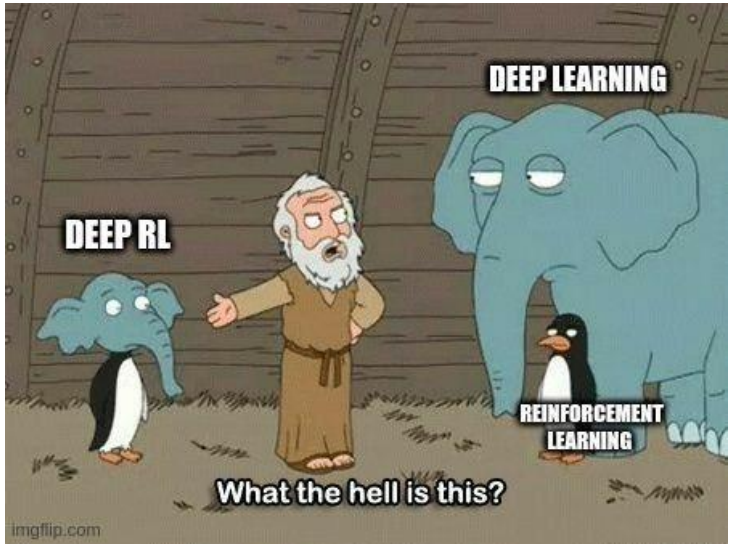
bootstrapping

+

sampling

Bias v.s. Variance

- MC has high variance, zero bias
  - Good convergence properties
  - Not very sensitive to initial value
  - Very simple to understand and use



- TD has low variance, some bias
  - Usually more efficient than MC
  - TD(0) converges to $V^{\pi}(s)$ (but not always with function approximation)
  - More sensitive to initial value

SARSA: On-Policy TD Control

S, A, R, S', A'

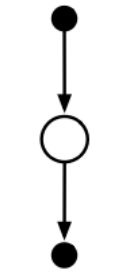
the name of SARSA comes from the transition from state S to S'

- Idea:
  - Continually estimate Q^π for the behavior policy π
  - At the same time change π toward greediness w.r.t. Q^π
- Algorithm:

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal



Sarsa

SARSA: On-Policy TD Control

- Convergence guarantee: SARSA converges conditions on the step sizes, as long as *all state-action pairs are visited an infinite number of times and the policy converges in the limit to the greedy policy*

  - All state-action pairs are explored infinitely many times:
    $$\lim_{k\to\infty} N_k(s,a) = \infty$$
  - The policy converges in the limit to the greedy policy:
    $$\lim_{k\to\infty} \pi_k(a|s) = \mathbb{I}(a = \text{argmax}_{a'\in A} Q_k(s,a'))$$
    - For example, ϵ-greedy is GLIE (greedy in the limit) policies by setting $\epsilon = \frac{1}{t}$ (ϵ reduces to 0)

Q-learning: Off-Policy TD Control

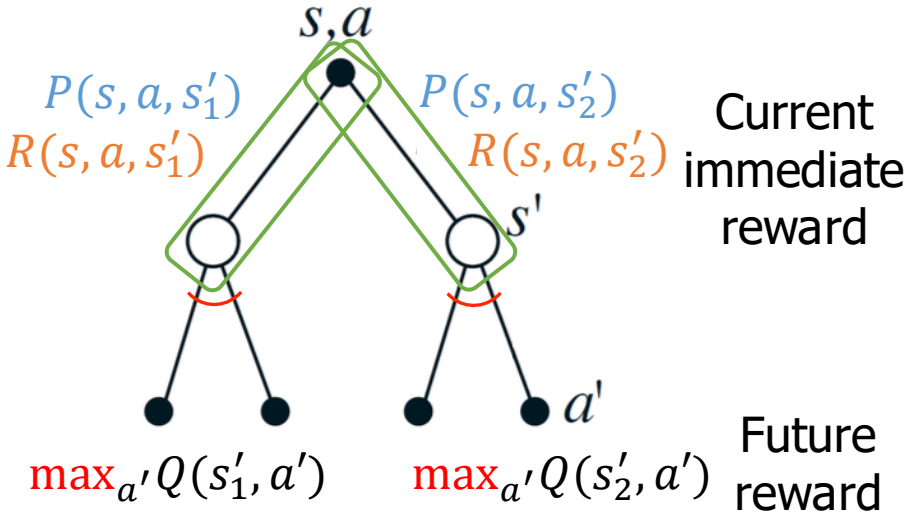
- Bellman optimality equation:

$$Q^*(s,a) = \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma \max_{a'} Q^*(s',a')\right]$$

- Q-value iteration: $Q_{k+1}(s,a) \leftarrow \sum_{s'} P(s'|s,a)\left[R(s,a,s') + \gamma \max_{a'} Q_k(s',a')\right]$

⇩ Rewrite as expectation

$$Q_{k+1} \leftarrow \mathbb{E}_{s' \sim P(s'|s,a)}[R(s,a,s') + \gamma \max_{a'} Q_k(s',a')]$$



Current immediate reward

Future reward

- Problem: cannot compute the update without the knowledge of P and R
- Idea: use actual samples to approximate the expectation
  - ↳ ~ drawn from P

Q-learning: Off-Policy TD Control

- Algorithm:

Q-learning: **sampled-based** Q-value iteration

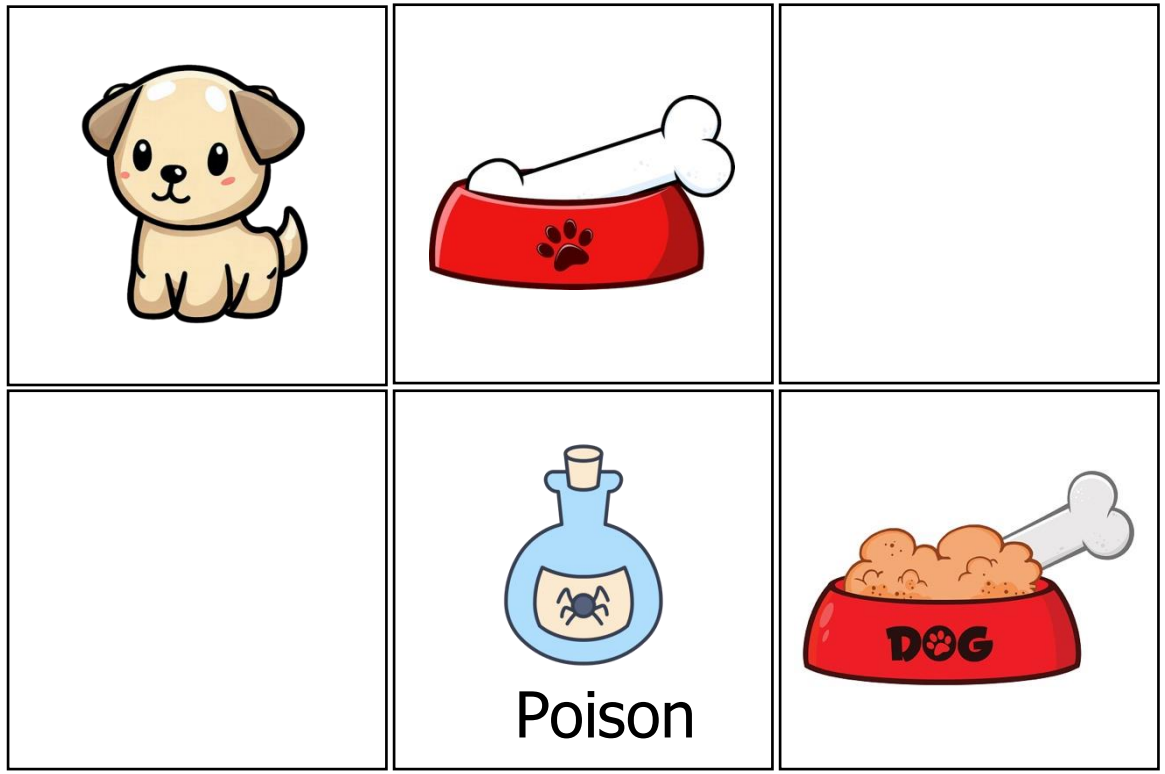
- Learn $Q(s,a)$ as you go
  - ➢ Receive a sample (s,a,r,s')
  - ➢ Compute the current sample: sample $= R(s,a,s') + \gamma \max_{a'} Q(s',a')$

    immediate reward; future returns
  - ➢ Update to $Q(s,a)$ by **running average**: $Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha$ sample

    learning rate; new estimate; old estimate; sampled estimated; weighted combination
  - ✓ Effect: Move values toward value of whatever successor occurs
  - ✓ Theoretical guarantee: Convergence

Q-learning: Off-Policy TD Control

- Example:



Reward
- +1: eat the small bone
- +10: eat the animal food with the bone
- -10: eat the poison
- 0: otherwise



Goal: eat the animal food with the bone while avoiding the poison

- Termination: if the dog eats the poison/animal food with the bone, or it spends more than 5 steps

Q-learning: Off-Policy TD Control

- Example:

Initialize the Q-table

Actions

States

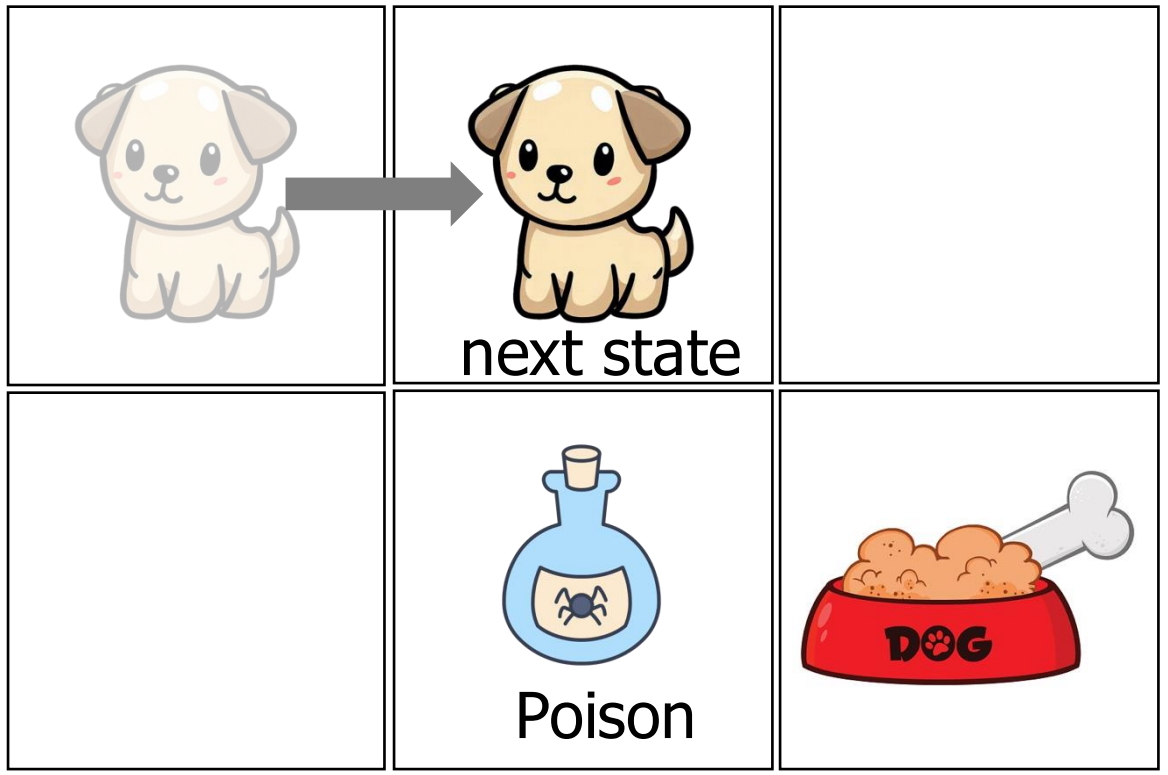
	↑	↓	←	→
(0, 0)	0	0	0	0
	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-learning: Off-Policy TD Control

- Example:
  - Choose action a based on a lookup of the Q-table
  - Obtain the next state and reward

	↑	↓	←	→
(0, 0)	0	0	0	0
	0	0	0	0
(0, 2)	0	0	0	0
(1, 0)	0	0	0	0
	0	0	0	0
	0	0	0	0

Q-table



Environment

Q-learning: Off-Policy TD Control

- Example:

Update the Q-value

Learning rate

$$Q(s,a) \leftarrow Q(s,a) + \alpha(R(s,a) + \gamma \max_{a'} Q(s',a') - Q(s,a))$$

New Q-value estimation | Previous Q-value estimation | Immediate reward | Discounted estimated optimal next Q-value | Previous Q-value estimation

Target value

$$Q(s=(0,0), a=\rightarrow) = 0 + 0.1 \times [1 + 0.99 \times 0 - 0] = 0.1 \quad (2)$$

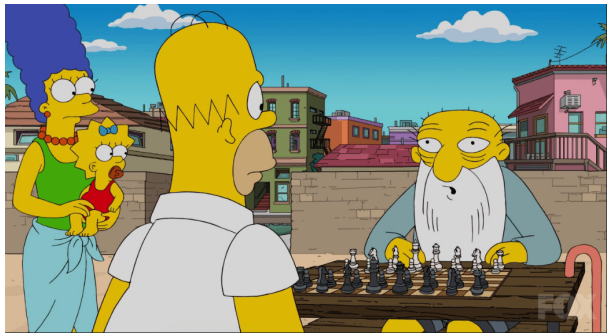
- learning rate $\alpha = 0.1$
- discount factor $\gamma = 0.99$

Q-learning: Off-Policy TD Control

- Amazing result: Q-learning converges to optimal policy -- even if you are acting sub-optimally!
- This is called off-policy learning

Behavior Policy: policy used to navigate in an environment

Target Policy: policy used to optimize decision making



On-policy: behavior policy should be the same as the target policy



Off-policy: behavior policy can be different from target policy

Q-learning: Off-Policy TD Control

- Amazing result: Q-learning converges to optimal policy -- even if you are acting sub-optimally!
- This is called off-policy learning
- Caveats:
  - You have to explore enough
  - You have to eventually make the learning rate small enough
  - … but not decrease it too quickly



Technical requirements:

❑ All states and actions are visited infinitely often
- Basically, in the limit, it does not matter how you select actions (!)

❑ Learning rate schedule such that for all state and action pairs (s, a):

$$\sum_{t=0}^{\infty} \alpha_t(s,a) = \infty \qquad \sum_{t=0}^{\infty} \alpha_t^2(s,a) < \infty$$

Q-learning: Off-Policy TD Control

Model-free Does not require the knowledge of the model

Convergence
- Every (s, a) is visited infinitely often
- Decaying learning rates condition

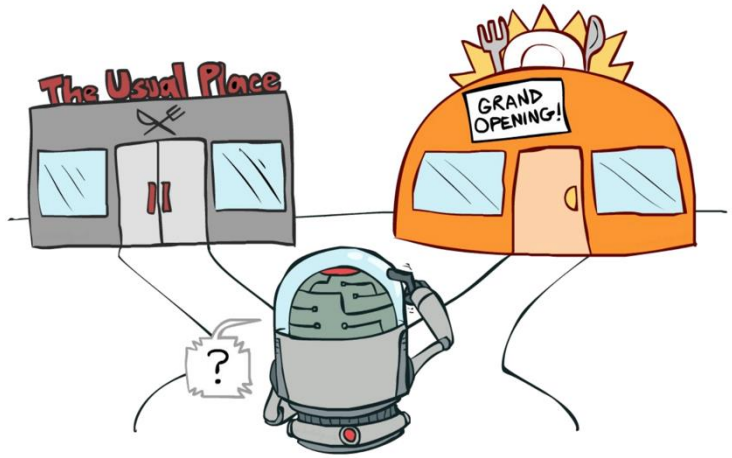
- Does not matter how actions are seletced
- **Exploration/exploitation strategy**
  - ϵ-greedy

    w.p. ϵ: act randomly (**exploration**)

    w.p. $1 - \epsilon$: act according to the current greedy policy (**exploitation**)



(Can lower ϵ over time)



Off-policy Sample-efficient

Checklist

- **Markov Decision Process (MDP)**
  - Core components
  - Understanding how MDPs formalize sequential decision problems
  - Bellman equations as recursive relationships between value functions
- **Value Functions and Optimality**
  - State-value functions and action-value functions
  - Optimal policy and its relationship to optimal value functions
  - Recognizing different Bellman operators from their mathematical forms
- **Dynamic Programming**
  - Value Iteration: update equation, convergence properties, computational complexity
  - Policy Iteration: policy evaluation and policy improvement steps, computational complexity
  - How discount factor affects optimal policies

Checklist

- **Monte Carlo vs. Temporal Difference Learning**
  - Monte Carlo methods: learning from complete episodes, unbiased but high variance
  - TD learning: bootstrapping from incomplete episodes, biased but lower variance
  - TD error calculation
- **Q-learning and SARSA**
  - Q-learning as an off-policy control algorithm
  - SARSA as an on-policy alternative
  - Update equations and implementation details
  - How exploration policies affect learning
- **Exploration Strategies**
  - ε-greedy approach to balancing exploration and exploitation
  - GLIE (Greedy in the Limit with Infinite Exploration) conditions
  - Impact of exploration parameters on learning performance